Digital Design Using Verilog HDL

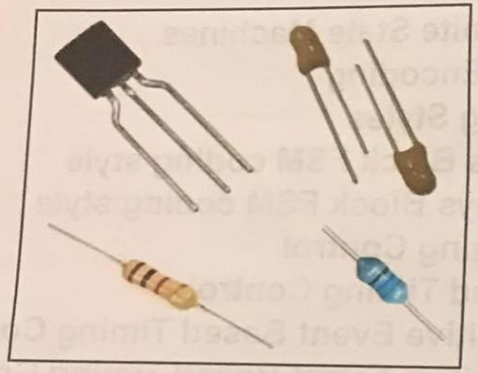
BY: Eng Ali Eltemsah

CONTENTS

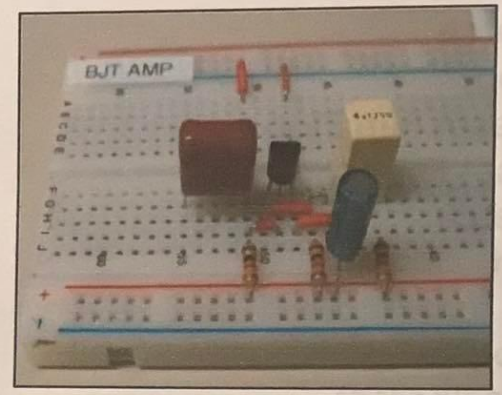
CONTENTS

What is an Electronic Circuit?

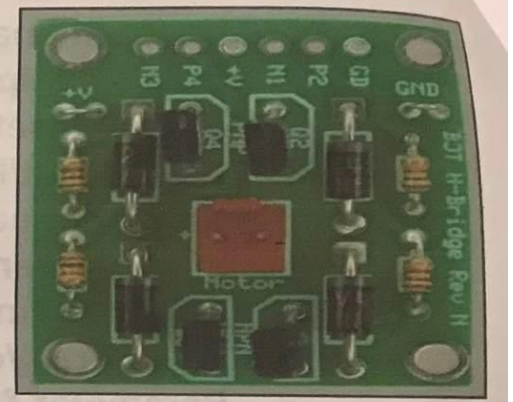
- Electronic circuit is composed of individual components, such as resistors, transistors, capacitors, inductors and diodes, connected by conductive wires to perform many functions such as signal amplification, computation, or data transfer.



Discrete Components



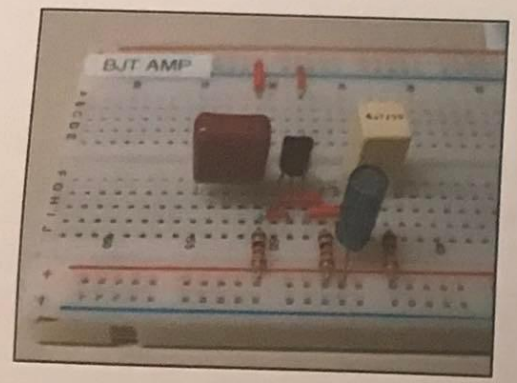
BreadBoard

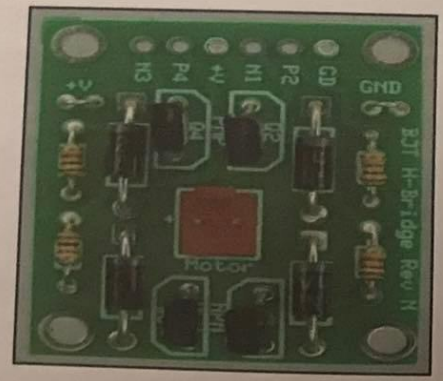


PCB

What is an Integrated Circuit (IC) ?

- *IC is an electronic circuit formed on a small piece of semiconducting material contains billions of transistors, performs the same function or more as a larger circuit made from discrete components.*





IC

Discrete Circuit Vs Integrated Circuit

1. **Area :** The integrated circuit has very small in size relative to discrete circuit. It is practically **billions** of electronic components can be integrated in a single square inch of IC chip.

2. **Functionality :** ICs nowadays can contains huge functions in a very small area compared to the discrete circuit.

3. **Reliability :** The integrated circuit is most reliable than the discrete circuit as the interconnects and the components are fabricated on the same chip with the same technology, while wires and components in the discrete circuit are usually connected with soldering process as bad solder joint add resistances on the circuit. soldering can reach to 400 degrees celsius and this high temperature can damage the components.

4. **Performance :** The Integrated circuits can reach to vey high operational speed because parasitic elements such as capacitors and resistance are omitted as well as ICs latest technologies consume very low power compared to discrete circuits.

5. **Replacement of failed components :** When a component of an IC is damaged, the whole IC should be replaced by a new one as it is impossible to replace or repair the component inside.

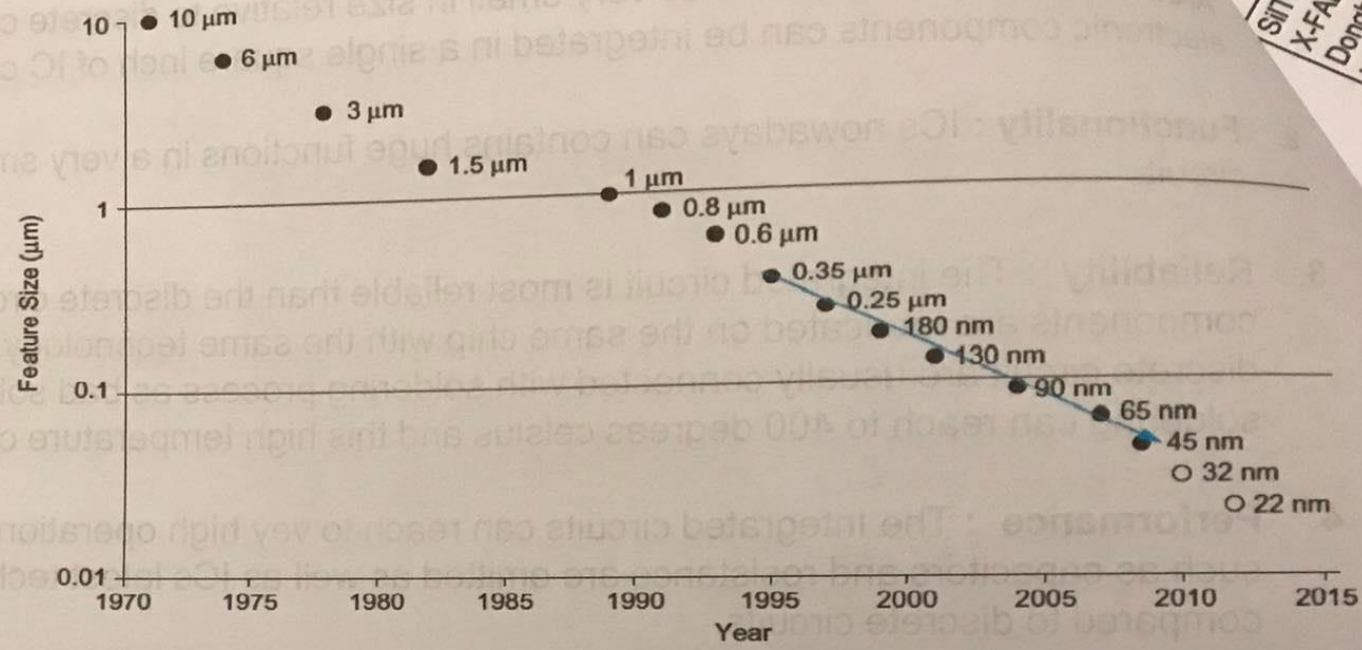
6. **Limitation of ICs :** Only small capacitors (in range of pF) and small resistors (in range of KOhms) can be integrated on-chip while large capacitors and large resistors are always integrated off-chip.

7. **Cost :** Production of IC's are done in bulk or mass production. Therefore, the price of IC per unit will be reduce due to high supply. It will be much inexpensive rather than creating equivalent discrete circuit of an IC.

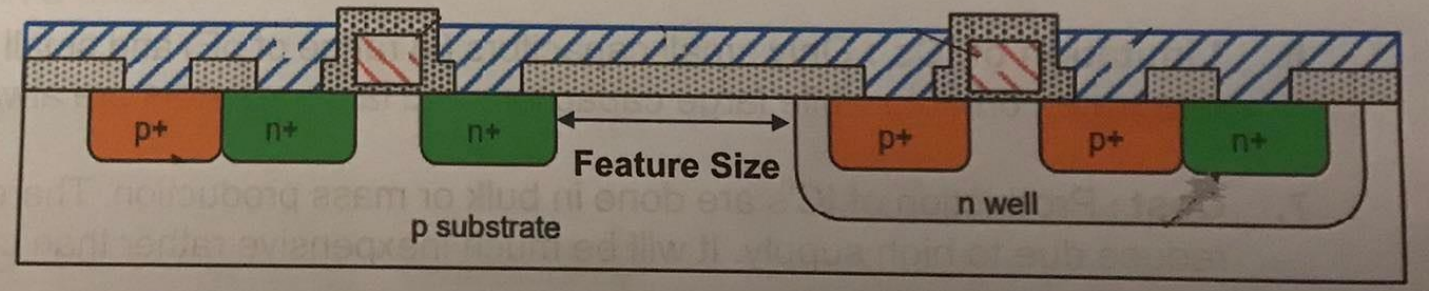
8. **Security**

Moore's Law & Technology Node

- **Moore's law [1965]: Moore's Law predicted that transistor count would double every 2 years due to shrinking transistor dimensions and other improvements but in fact that the transistor size has decreased by a factor of two every 18 months (1.5 Year). Device scaling brings new opportunities & challenges**



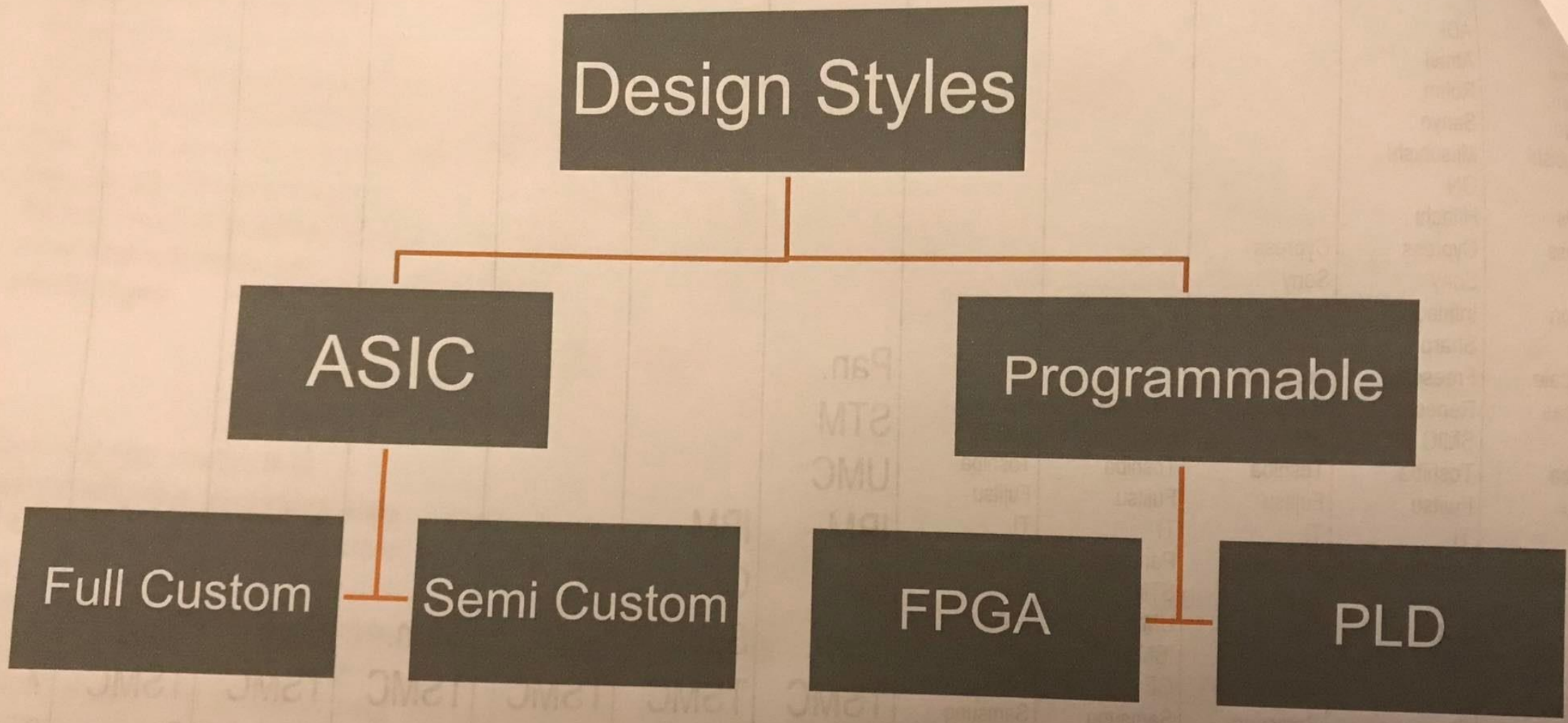
- ***Technology node: it is essentially the physical size of the transistor, the smaller the transistors, the more transistors in the same area, the faster they switch, the less energy they require and the cooler the chip runs.***



Foundries with a Cutting-Edge Logic Fab

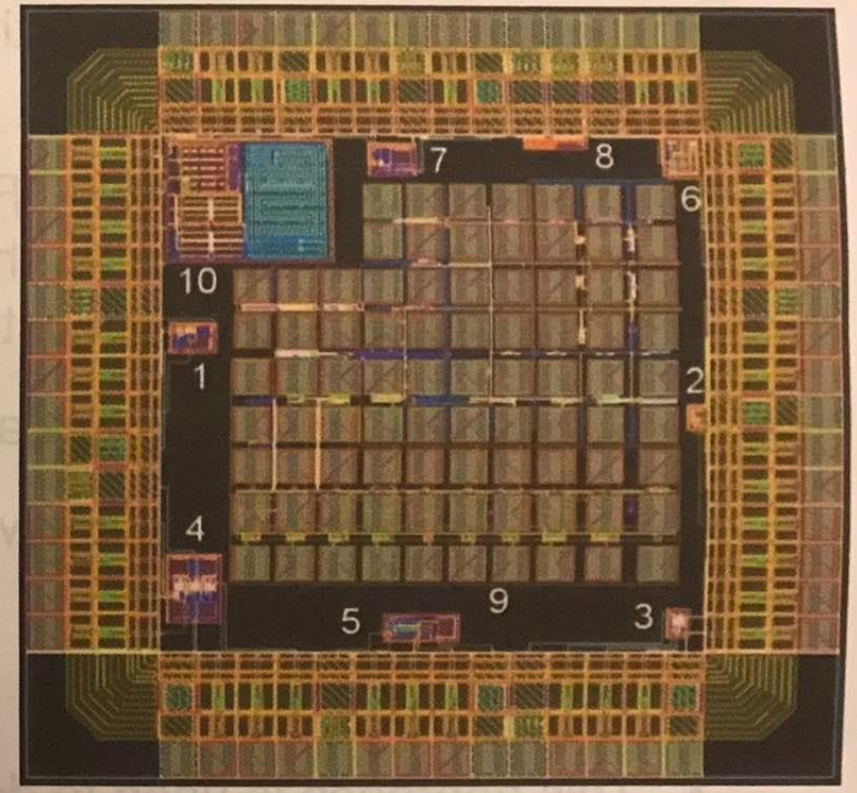
SilTerra X-FAB Dongbu HiTek ADI Atmel Rohm Sanyo Mitsubishi ON Hitachi Cypress Sony Infineon Sharp Freescale Renesas SMIC Toshiba Fujitsu TI Panasonic STM UMC IBM AMD Samsung TSMC Intel	ADI Atmel Rohm Sanyo Mitsubishi ON Hitachi Cypress Sony Infineon Sharp Freescale Renesas SMIC Toshiba Fujitsu TI Panasonic STM UMC IBM AMD Samsung TSMC Intel	Cypress Sony Infineon Sharp Freescale Renesas SMIC Toshiba Fujitsu TI Panasonic STM UMC IBM AMD Samsung TSMC Intel	Renesas SMIC Toshiba Fujitsu TI Panasonic STM UMC IBM GF Samsung TSMC Intel	Renesas SMIC Toshiba Fujitsu TI Panasonic STM UMC IBM GF Samsung TSMC Intel	Pan. STM UMC IBM GF Sam. TSMC Intel	IBM GF Sam. TSMC Intel	GF Sam. TSMC Intel	Sam. TSMC Intel	Sam. TSMC Intel	Sam. TSMC ?	? ? ?
180 nm	130 nm	90 nm	65 nm	45/40 nm	32/28 nm	22/20 nm	16/14 nm	10 nm	7 nm	5 nm	3 nm

VLSI Design Styles



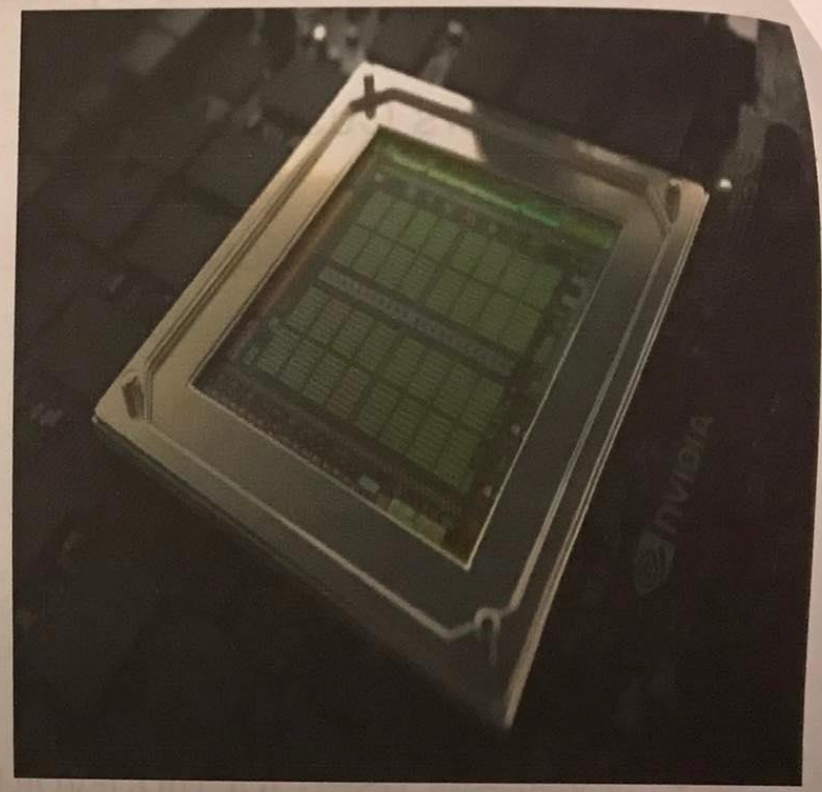
Full Custom Design

- Compete design, layout and placement of the transistor is done by the designer.
- All Layers Customized
- Huge design effort
- Long Design Time
- High Performance
- Simulation at transistor level
- Typically used for high-volume applications
- Example : AND Gate IC



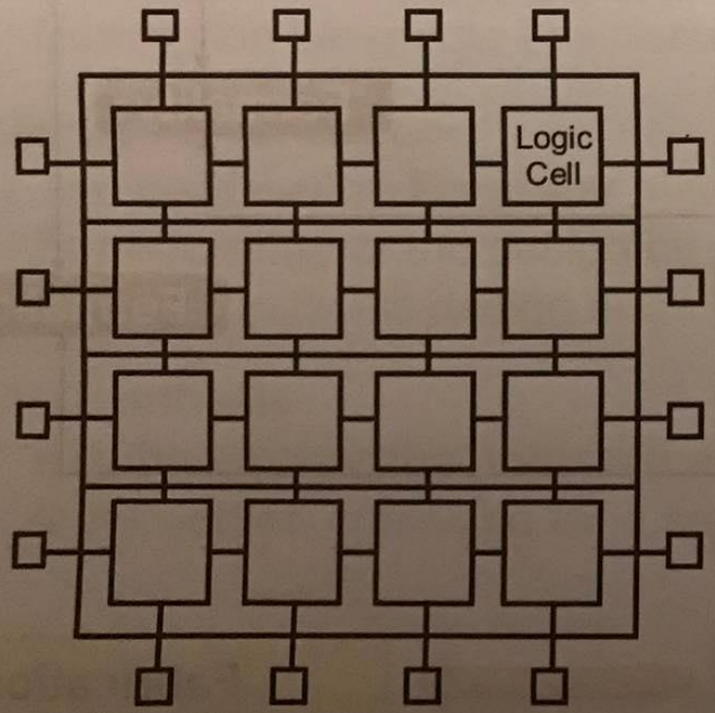
Semi Custom Design

- It is known as Standard Cell based ASIC.
- Design is completed with the use of standard cell libraries.
- The main Gates (AND, OR, Flip Flops, ..) known as standard cells, these cells are predesigned using full custom approach
- Reasonable design effort and time
- Good Performance but low compared to full custom approach
- Simulation at gate level
- This approach is used to design ICs to fit a certain application
- Example : Processors, GPUs .



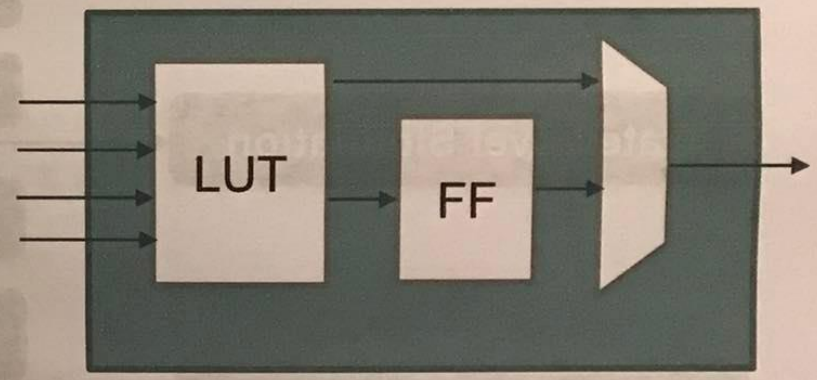
FPGA Design

- Field Programmable Logic Array (FPGA)
- Integrated Circuit designed to be configured by the designer using a Hardware Description language HDL
- It is mainly used for “Prototyping” to reduce time to market.
- The FPGA contains Logic Cells, so the transistor level is not considered.

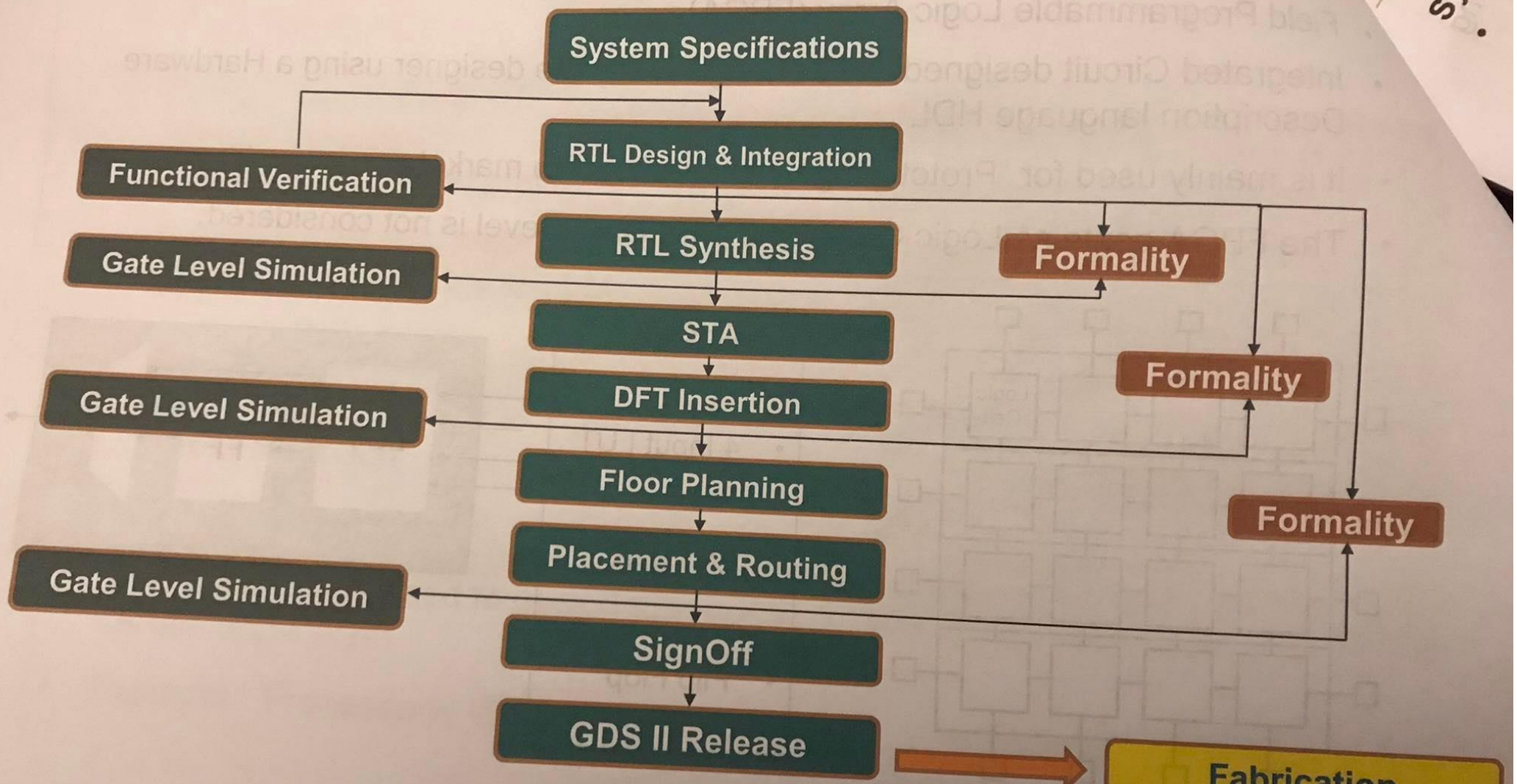


Logic Cell

- 4 Input LUT
- 2X1 Mux
- Flip Flop



ASIC (Semi Custom) Design Flow



ASIC Design Flow

System Specifications(specs)

- The chip functionality is described in "Requirement & Specification Document" by the **system engineer** according to agreement with the customer.

RTL Design

- The **Design Engineer** start to put the architecture of the design to meet the system requirements (Functional, Interfacing with other blocks, operational speed, power consumption, Area) and then the architectural micro designs are modelled in a Hardware Description Language like Verilog/VHDL, using synthesizable constructs of the language.

Functional Verification

- The **Verification Engineer** start to build the verification environment to test the **Functionality** of the design by simulating all the test cases and report to the design engineer to fix his design bugs.

RTL Synthesis

- It is the process that translate and map the HDL code into a technology dependent **gate-level netlist**, optimized for a set of pre-defined constraints.

Static Timing Analysis (STA)

- It is a method of checking the ability of the design to meet the **timing requirements** as setup, hold, Recovery, Removal timing checks.

ASIC Design Flow

DFT Insertion

- Extra hardware is added to detect any manufacturing defects introduced to the chip during fabrication.

Formality (Formal Verification)

- It is the process that verify the **functionality equivalence** of the **gate netlist** in different stages against the **RTL** using formal verification (Mathematical) techniques

Floor planning

- It is the process that determine Die Size, blocks locations, power planning and I/O Pad's location (Pin Placement).

Placement & Routing

- It is the process that place the cells on the chip and plan routing between blocks that target minimizing the total interconnect length which is the final step before generation of layout.

GLS (Gate Level Simulation)

- It is the process that verify both **timing** and **functionality** of the generated gate level netlist after both logic synthesis and PnR steps.

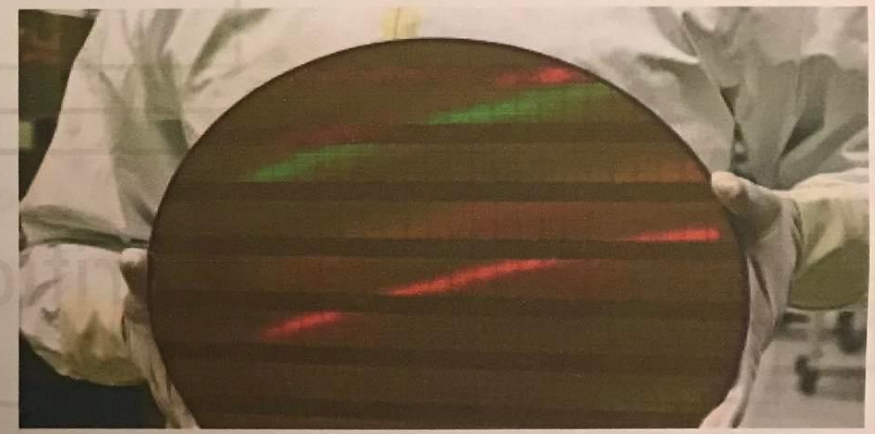
SignOff & GDS II Generation

- Signoff is the process that check LVS (Layout Verses Schematic), DRC (Design Rule Constraint) and Timing Closure of the final netlist then finally generate the IC layout in form of GDSII stream format (industry standard for data exchange of IC layout).

Production

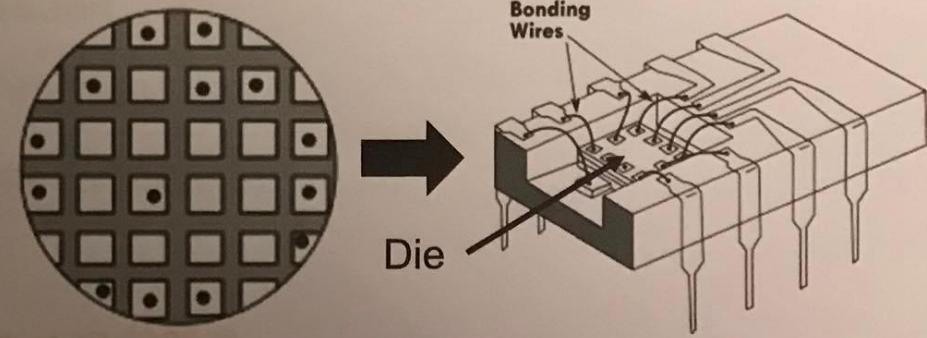
Fabrication

- Chips are built in huge factories called fabs (fabrication plants)
- Fabs contain “clean rooms” which clean of dust that classified by “class 100”, “class 10”.
- Fabrication process is very expensive as processing 300 mm wafers in a 45 nm process costs about $3 billion (that’s why many semiconductor companies are fabless)
- Big players: Intel, Samsung, TSMC, Global Foundries, …
- The wafer is cut (diced) into many pieces, Each of these pieces is called a die

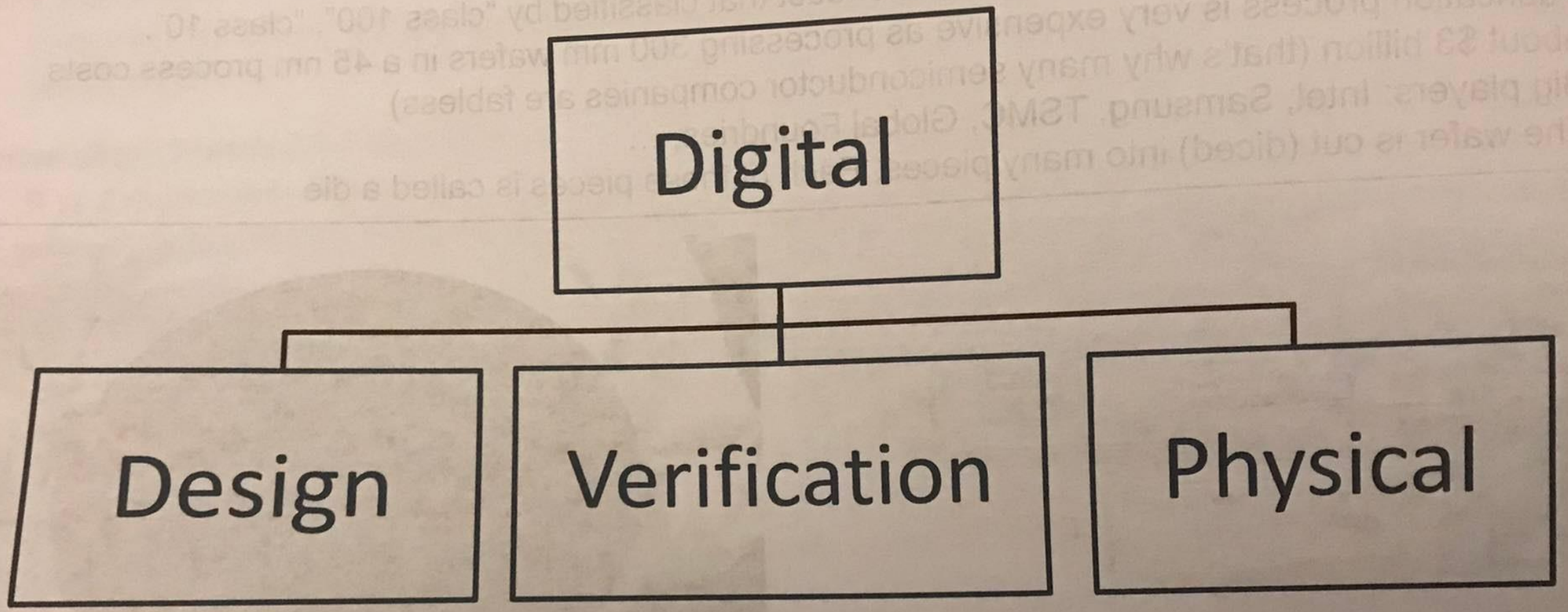


Packaging

- The Die need to be packaged to:-
  - Protect chip from mechanical damage
  - Removes heat produced on the chip
  - Connect chip to board easily



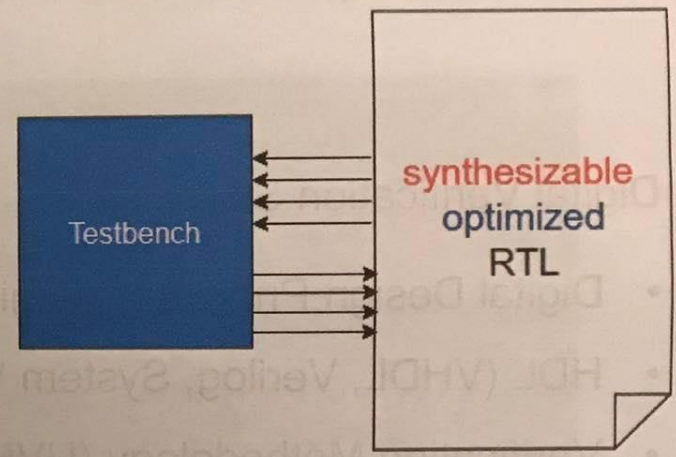
Digital Engineers Careers



Digital Design Engineer

Digital Designer role:-

- Digital Design Engineer is responsible for writing a synthesizable & optimized RTL and generate stimulus to test and monitor the response of the design for proper functionality.



Digital Design Skills:-

- Digital Design Principals (Logic, MP, DSP,VLSI)
- Knowledge of HDL (VHDL, Verilog).
- Knowledge of Testbenches & RTL simulation
- Knowledge of Clock Domain Crossing Issues
- Knowledge of RTL Power reduction techniques
- Familiar with RTL synthesis & Gate Level Simulation (GLS)

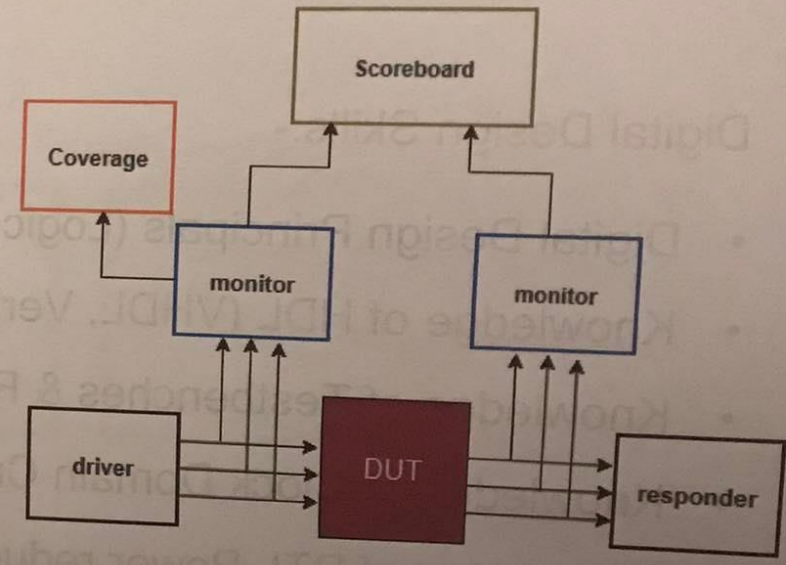
Digital Verification Engineer

Digital Verification role:-

- Verification engineer is responsible for building a verification environment, developing testing plans and implement testing procedures to determine if products work as intended or not.

Digital Verification Skills:-

- Digital Design Principals (Logic, MP, DSP)
- HDL (VHDL, Verilog, System Verilog).
- Verification Methodology (UVM, OVM)
- Programming Data Structures, OOP Concepts
- Scripting Language (TCL, Perl)



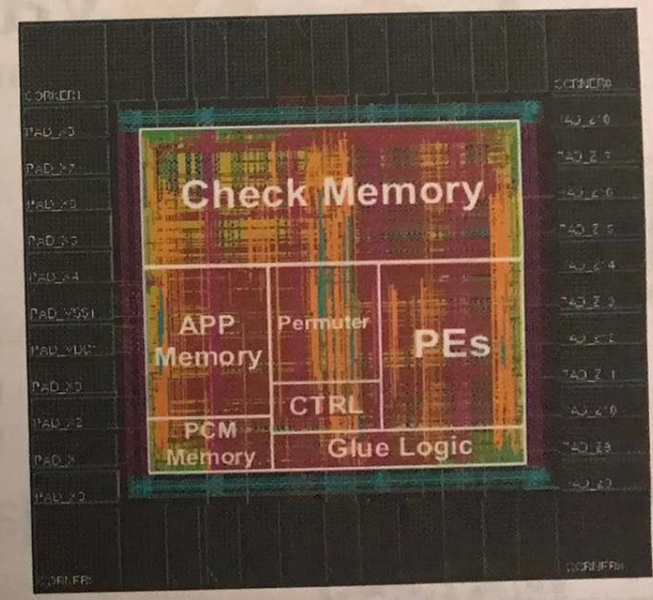
Physical Designer Engineer

Physical Designer role:-

- Physical Design engineer is responsible for implementing the design floor planning, pin placement, and power planning, cells placement and routing, perform static timing analysis and timing closure, DFT Insertion and GDS II Generation.

Physical Designer Skills:-

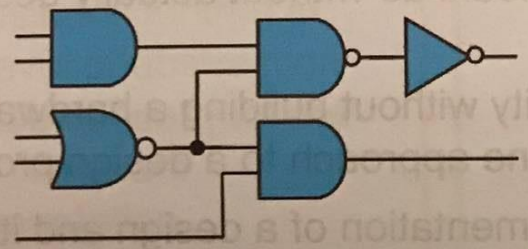
- Solid understanding of ASIC Flow
- Familiar with RTL synthesis & GLS
- STA and timing closure Methodologies
- Knowledge of DFT Insertion
- Scripting Language (TCL, Perl)



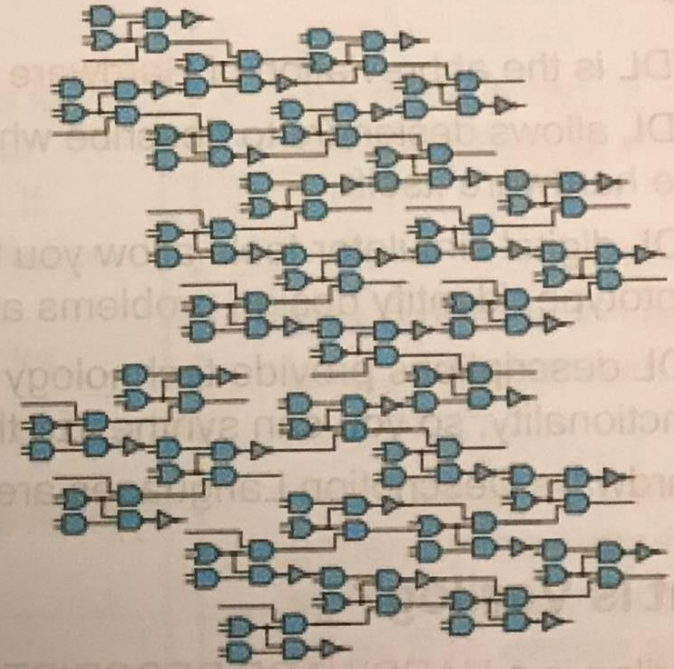
VLSI Industry in Egypt

Introduction to Verilog HDL

Why HDL?



In the beginning designs involved just a few gates, and thus it was possible to verify these circuits on paper or with breadboards



When designers began working on 100,000 gate designs, these gate-level models were too low-level for the initial functional specification and early high-level design exploration

Introduction to Verilog HDL

Why HDL?

- HDL is the abbreviation of Hardware Description Language.
- HDL allows designers to describe what the hardware should do without actually designing the hardware itself.
- HDL digital simulator tools allow you to verify functionality without building a hardware prototype, identify design problems and try more than one approach to a design problem.
- HDL descriptions provide technology independent documentation of a design and its functionality, so you can synthesize the same design into different technologies.
- Hardware Description Languages are VHDL, Verilog and System Verilog.

What is Verilog?

- Verilog is a HARDWARE DESCRIPTION LANGUAGE.
- Not a programming language despite the syntax being similar to C
- Verilog are used to describe digital circuits.
- Verilog is a Case sensitive Language.
- Verilog are synthesized to give the circuit logic diagram
- Verilog is a single language used for design and simulation
- Verilog used to describe a design at different levels of abstraction (Behavioral level, RTL level, Gate (Structural) level).

Verilog abstraction Levels

Behavioral Level

- It is a model that describe the design algorithm in high-level of abstraction with little concern for the actual hardware implementation.
- Easier to write and understand.
- Not always synthesizable!!!

RTL Level

- RTL design comes in between behavioral level & Gate level. It has detailed description of the circuit with respect to the clocks and data flow
- It has intermediate level of abstraction
- synthesizable

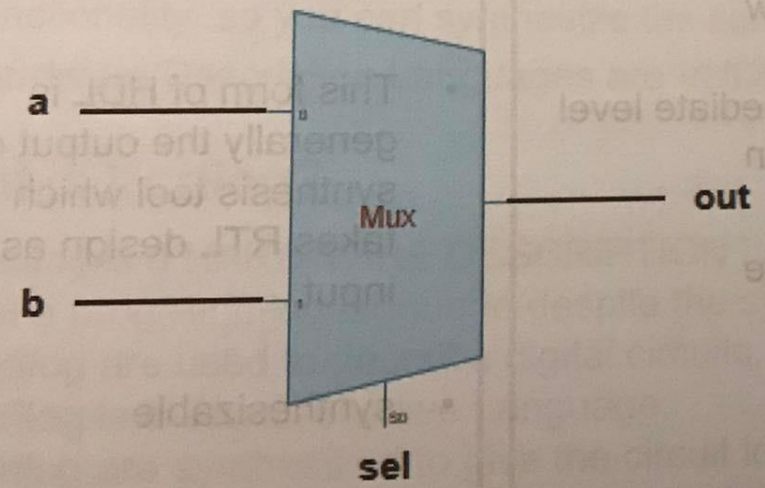
Gate Level

- This is the physical representation of a circuit
- This model hasn't any level of abstraction.
- This form of HDL is generally the output of a synthesis tool which takes RTL design as input.
- synthesizable

Verilog abstraction Levels

RTL (Register Transfer Level)

- It is a model that describe the flow of data between registers and how a design processes that data by describing the behavior of the circuit
- Example: 2 input Multiplexer.

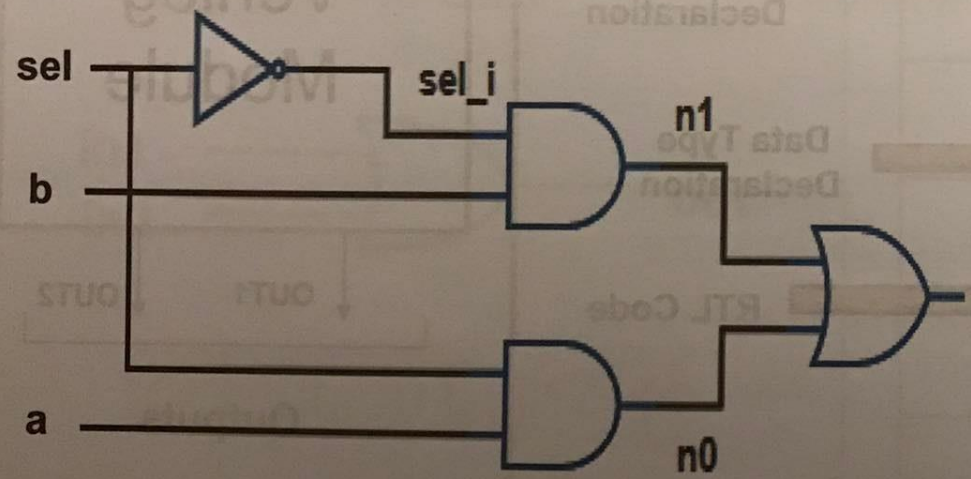



```
module mux2( a, b, sel, out );

input a, b, sel ;
output out ;

 always @ (a,b,sel)
  begin
   if(sel==0)
    out = a ;
   else if (sel==1)
    out = b ;
  end

endmodule
```

Verilog abstraction Levels

Gate Level (Structural)

- Design is implemented in terms of basic logic gates (AND, OR, NOT) and their interconnections.
- Example: 2 input Multiplexer.




```
module mux2( a, b, sel, out );

input a, b, sel ;
output out ;

wire  sel_i;
not not0( sel_i, sel );

wire n0, n1 ;
and and0( n0, a, sel );
and and1( n1, b, sel_i ) ;

or or0( out, n0, n1 ) ;

endmodule
```

We will use a mix of RTL (~95%) and gate-level (~5%)

Module structure & Port Declaration 95

```
Module Full_Adder
( IN1, IN2, CLK, RST, OUT1, OUT2 );

Input     [7:0]    IN1, IN2 ;
Input              CLK, RST;
Output             OUT;

reg   [3:0]  sig1;
wire         sig2;

// Describe Your Circuit

endmodule
```

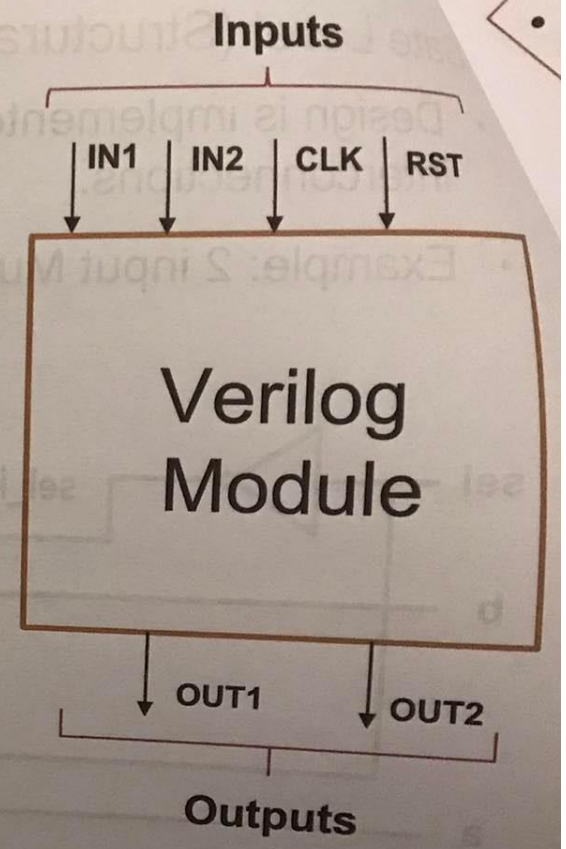
Module Name

Port List

Port Declaration

Data Type Declaration

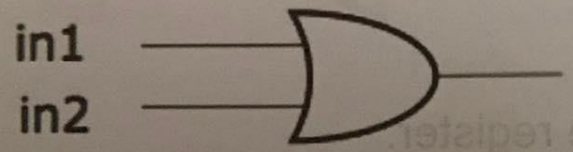
RTL Code



Verilog Primitives

- Verilog provide some primitives of the basic combinational logic gates that helps in the gate level design.
- Examples:-

1. Two Input OR Gate




```
or (out, in1, in2) ;
```

output Input Input

Primitive Name	Functionlity
and	Logical And
or	Logical And
not	Inverter
buf	Buffer
xor	Logical Exclusive Or
xnor	Logical Exclusive Nor
nand	Logical Inverted And
nor	Logical Inverted Or

Signals Types

wire

- It describe a simple physical connection between two structural elements.
- Any assignments using assign statement must assign to a wire variable
- All inputs and output ports by default of wire type
- It is 4 states : - 0 (Logic 0) - 1 (Logic 1)
  - X (Don't care) - Z (High Impedance)
- Ex: wire [7:0] BYTE; // declare 8-bit signal named BYTE of type wire

reg

- A variable that may or may not represent a hardware register.
- Any assignments in an always block must assign to a reg variable.
- Used for applying stimulus in testbench
- It is 4 states : - 0 (Logic 0) - 1 (Logic 1)
  - X (Don't care) - Z (High Impedance)
- Ex: reg [15:0] INFO; // declare 16-bit signal named INFO of type reg

always block & sensitivity list

- Syntax

```
always @ (sensitivity list)
  begin
    statements;
  end
```

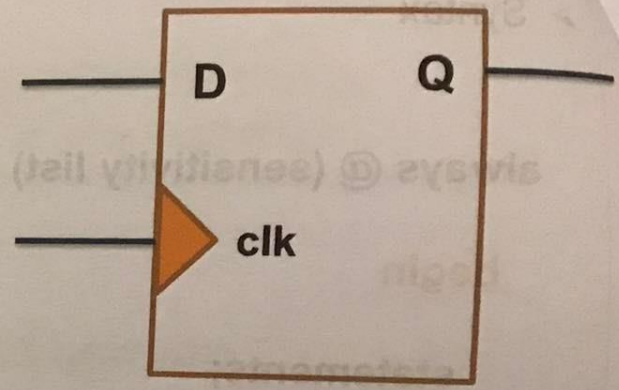
Don't forget semicolon

- always block is one of procedural blocks; statements inside always block executed in sequence (procedurally).
- The code inside always block executes whenever one of the variables in the (sensitivity list) changes.
- begin … end only needed if more than one statement in block.
- Used to describe combinational and sequential circuits
- All the always blocks are executed in parallel.
- All the variables assigned inside are of type reg

How to describe edge triggered Flip Flop

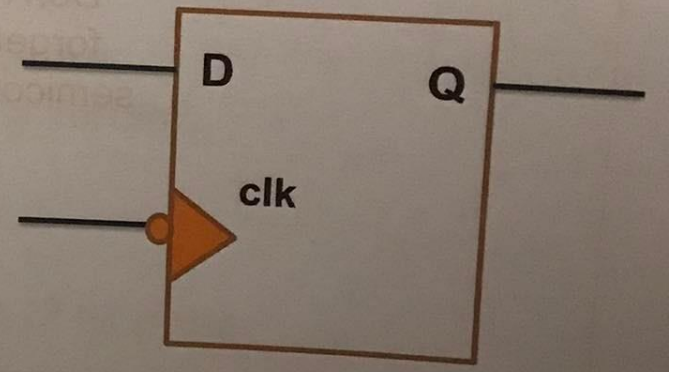
```
reg  Q ;

    always @ (posedge clk)
      begin
        Q <= D ;
      end
```

Q re-evaluated every time there is a rising edge of the clock and remains unchanged between rising edges




```
reg  Q ;

    always @ (negedge clk)
      begin
        Q <= D ;
      end
```

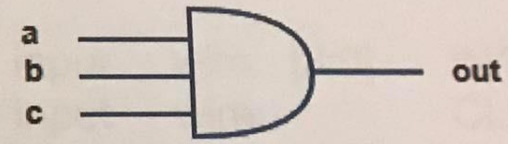
Q re-evaluated every time there is a falling edge of the clock and remains unchanged between falling edges



How to describe Combinational logic

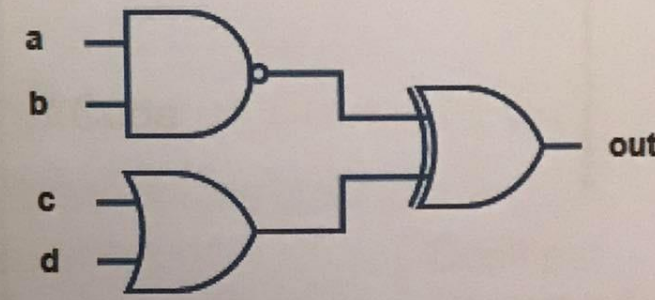
```
reg  out ;

     always @ (a or b or c)
      begin
        out = a & b & c ;
      end
```

3-input AND




```
reg  out ;

     always @ (a or b or c or d)
      begin
       out = (~(a&b)) ^ (c|d) ;
      end
```

complex logic



Tips

Each module should be in a separate file.

Name of the file is <modulename>.v

Write each input/output port on a separate line.